

Generative Diffusion Model Enhanced MADRL for Distributed Flocking With Obstacle Avoidance of Fixed-Wing AAV Swarms

Chao Yan , Member, IEEE, Yuna Jiang , Member, IEEE, Mou Chen , Fellow, IEEE, Xiaojia Xiang , and Xiangke Wang , Senior Member, IEEE

Abstract—The distributed flocking control of fixed-wing autonomous aerial vehicle (AAV) swarms in unknown and cluttered environments poses significant challenges due to their complex dynamics and nonholonomic constraints. While multiagent deep reinforcement learning (MADRL) has shown success in coordinating AAVs, existing approaches often struggle to capture the multimodal action distributions essential for robust obstacle avoidance under uncertainty. In this article, we propose a novel generative diffusion model enhanced MADRL approach for distributed flocking control and obstacle avoidance in fixed-wing AAV swarms. Specifically, we design a diffusion-based policy network, where individual observations are processed via a graph attention network and used as conditions for generating optimal actions in the reverse denoising process. To update the diffusion policy, a graph attention network is also incorporated into the value network to effectively extract relevant information from all agents, thereby improving the accuracy of the Q-value estimation. The results of numerical simulations and hardware-in-the-loop experiments demonstrate that our approach significantly outperforms existing baselines in terms of flocking stability and collision rate, highlighting the potential of diffusion models for scalable and robust AAV flocking control.

Index Terms—Fixed-wing autonomous aerial vehicle (AAV) swarm, flocking control, generative diffusion model (GDM), multiagent deep reinforcement learning (MADRL), obstacle avoidance.

I. INTRODUCTION

Autonomous aerial vehicle (AAV) swarms show immense promise across a wide range of critical applications, from security patrol [1] and emergency communication [2] to urban air mobility [3] and advanced air mobility [4], [5]. Hence, cooperative control techniques that coordinate a swarm of individual AAVs to accomplish complex tasks in a decentralized manner have become a hot research topic [6], [7]. Among the various approaches to cooperative control, distributed flocking offers a scalable and decentralized solution for maintaining formation coherence [8], [9], while obstacle avoidance ensures mission safety in complex environments. However, achieving flocking control with obstacle avoidance remains a huge challenge for AAV swarms, particularly when the sensing capability of each AAV is limited.

Traditional methods for achieving flocking and obstacle avoidance in AAV swarms, such as those based on Reynolds' rules [10], [11] or potential field methods [9], [12], often suffer from several limitations. These methods typically rely on handcrafted rules and parameters that can be challenging to tune optimally for complex and unknown environments. Furthermore, traditional methods can be brittle when faced with unforeseen situations, such as novel obstacle configurations or environmental changes. These methods often struggle with adaptability, resulting in issues, such as convergence to local minima, oscillatory behaviors, or even collisions. These challenges motivate the adoption of more adaptive, learning-based approaches that can continuously optimize swarm behavior in response to environmental changes.

In recent years, deep reinforcement learning (DRL), including multiagent deep reinforcement learning (MADRL) [13], [14], has become increasingly popular because of its superiority in model-free, flexibility, and computational costs [15]. Many researchers have successfully applied DRL for flocking control. For example, Shi et al. [16] designed a flocking controller for multiple AAVs in network-constrained environments by integrating heuristic guides with the twin delayed deep deterministic policy gradient (DDPG) algorithm. Xiao et al. [17] proposed the multiagent cooperative soft actor-critic algorithm for flocking control with obstacle avoidance under stochastic communication conditions. Jia et al. [18] developed an asymmetric self-play-empowered DRL framework for robust multirobot flocking

Received 22 May 2025; revised 14 October 2025; accepted 27 December 2025. Recommended by Technical Editor Z. Ji and Senior Editor H. Fujimoto. This work was supported in part by the National Natural Science Foundation of China under Grant 62403240, Grant 62533012, and Grant T2521006 and in part by the Natural Science Foundation of Jiangsu Province under Grant BK20241396. (Corresponding author: Mou Chen.)

Chao Yan and Mou Chen are with the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China (e-mail: yanchao@nuaa.edu.cn; chenmou@nuaa.edu.cn).

Yuna Jiang is with the School of Communications and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing 210003, China (e-mail: yunajiang@njupt.edu.cn).

Xiaojia Xiang and Xiangke Wang are with the College of Intelligence Science and Technology, National University of Defense Technology, Changsha 410073, China (e-mail: xiangxiaojia@nudt.edu.cn; xkwang@nudt.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TMECH.2026.3652629>.

Digital Object Identifier 10.1109/TMECH.2026.3652629

control in dynamic scenarios with environmental interferences. However, the above-mentioned studies only focus on particle-based agents, such as rotary AAVs or omnidirectional robots, which can hover or maneuver freely. Consequently, their methods are not applicable for fixed-wing AAVs that operate under more restrictive dynamics [19].

Fixed-wing AAVs present unique challenges due to their nonholonomic motion constraints, high-speed dynamics, and limited maneuverability [20], [21]. As a result, learning reliable flocking and obstacle avoidance policies becomes significantly more complicated. DRL has shown promise in achieving cooperative control of fixed-wing AAVs in related tasks. For example, Ma et al. [22] developed a DDPG-based control framework, enabling fixed-wing AAVs to adapt to complex formations while avoiding collisions. Zhuang et al. [23] proposed a proximal policy optimization based autonomous penetration method for high-speed fixed-wing AAVs against multiple interceptors. However, few efforts have been dedicated specifically to flocking control for fixed-wing AAV swarms. Notable exceptions include our previous work [24], [25], where we designed a multiagent DDPG based distributed flocking framework with attention mechanisms for fixed-wing AAVs in cluttered environments. Nevertheless, the aforementioned methods typically rely on either deterministic policies or unimodal Gaussian policies. This reliance limits their ability to learn diverse and adaptive flocking behaviors in complex environments, especially when multiple feasible paths or maneuvers are available around obstacles.

Motivated by these limitations, we turn our attention to generative diffusion models (GDMs) as a potential solution. Diffusion models have recently emerged as powerful generative models capable of capturing complex, multimodal distributions by progressively denoising random noise into structured outputs [26], [27]. Unlike unimodal Gaussian policies, diffusion models naturally support multimodal action generation, enabling agents to explore and exploit multiple plausible behaviors conditioned on the environment [28], [29]. This property is particularly advantageous for flocking control of fixed-wing AAV swarms, where AAVs may need to consider diverse maneuver options in the presence of dynamic obstacles and limited maneuverability.

In this article, we propose a novel GDM enhanced MADRL approach for distributed flocking control and obstacle avoidance of fixed-wing AAV swarms. The core insight behind the proposed approach is the design of a generative diffusion policy model that conditions on local observations and generates diverse, coordinated actions for each AAV in a decentralized manner. By integrating the generative capability of diffusion models into the policy learning process, our approach enables fixed-wing AAVs to maintain cohesive flocks while navigating safely through complex, cluttered environments. Extensive experiments validate the superiority of our approach over existing MADRL baselines in terms of flocking stability and collision rate.

To the best of our knowledge, we are the first to address the distributed flocking control problem for a scalable fixed-wing AAV swarm in unknown and obstacle-rich environments using the GDMs. The main contributions of this article are summarized as follows.

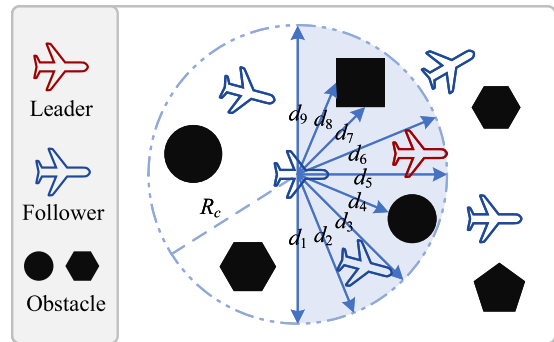


Fig. 1. Scalable swarm of fixed-wing AAVs consisting of one unique leader and n identical followers. $d_1, \dots, d_9$ denote the distances between the follower and the surrounding obstacles in nine directions measured by the distance sensor.

- 1) A novel multiagent actor-critic reinforcement learning approach enhanced by a GDM is proposed for achieving robust distributed flocking control and effective obstacle avoidance with a scalable fixed-wing AAV swarm.
- 2) A generative diffusion policy model is designed to generate diverse and coordinated actions, where local neighbor information is processed by a graph attention network (GAT) and utilized as the conditional input to the diffusion model.
- 3) Hardware-in-the-loop (HITL) experiments and extensive numerical simulations are conducted to comprehensively validate the effectiveness and robustness of the proposed approach for flocking control.

The rest of this article is organized as follows. Section II formulates the distributed flocking control and obstacle avoidance problem. Section III proposes the GDM enhanced MADRL approach, followed by the experiment results in Section IV. Finally, Section V concludes this article.

II. PROBLEM FORMULATION

In this section, we first describe the distributed flocking and obstacle avoidance problem for a scalable swarm of fixed-wing AAVs, and then formulate it as a decentralized partially observable Markov decision process (Dec-POMDP).

A. Problem Statement

In the considered flocking scenario, a scalable swarm of fixed-wing AAVs operates within a 2-D environment cluttered with multiple obstacles (see Fig. 1). The swarm consists of a unique leader followed by a dynamic set of identical followers denoted by $\mathcal{I} := \{1, 2, \dots, n\}$, where the number of followers n is variable. The state of the follower i can be described by the tuple $\xi_i := (x_i, y_i, \psi_i, \phi_i, v_i)$, representing the plane coordinates, heading angle, roll angle, and airspeed, respectively. Similarly, the leader's state can be denoted as $\xi_l := (x_l, y_l, \psi_l, \phi_l, v_l)$. The objective is to design a distributed flocking control policy that allows a flexible number of followers to flock with the leader while actively avoiding collisions with obstacles.

We assumed that all followers are equipped with identical and limited sensing capabilities. Each follower can only perceive the states of nearby agents within its sensing range R_c . Accordingly, the neighbor set of follower i is defined as

$$\mathcal{N}_i := \{j \mid \rho_{i,j} < R_c, j \in \mathcal{I}, j \neq i\} \quad (1)$$

where $\rho_{i,j}$ is the distance between follower i and follower j . In addition, each follower carries a nine-beam sparse distance sensor covering a 180° field-of-view to detect nearby obstacles. Denote the information measured by the distance sensor of follower i as

$$\mathbf{d}_i := [d_{i,1}, \dots, d_{i,m}, \dots, d_{i,9}] \quad (2)$$

where $d_{i,m}$ is the distance in the m th beam direction.

B. Dec-POMDP Model

1) **Observation Space:** All the followers have the same observation space. The observation of each follower $i \in \mathcal{I}$ can be defined as

$$\mathbf{o}_i := (\xi_l, \xi_i, \xi_{\mathcal{N}_i}, \mathbf{d}_i) \quad (3)$$

where ξ_l is the leader's state; ξ_i is the follower i 's state; $\xi_{\mathcal{N}_i} := \{\xi_j \mid j \in \mathcal{N}_i\}$ is the set of states of follower i 's neighbors; $\mathbf{d}_i$ indicates the distance information measured by the follower i .

2) **Action Space:** All follower share the same continuous action space. For each follower i , the action is defined as $\mathbf{a}_i = (\omega_i, u_i)$, where $\omega_i \in [-\omega_{\max}, \omega_{\max}]$ represents the roll command, and $u_i \in [-u_{\max}, u_{\max}]$ denotes the speed command. These high-level control commands are subsequently executed by the AAV's low-level autopilot according to the following dynamics [24], [30]:

$$\dot{\xi}_i = \frac{d}{dt} \begin{Bmatrix} x_i \\ y_i \\ \phi_i \\ v_i \end{Bmatrix} = \begin{Bmatrix} v_i \cos \psi_i + \eta_x \\ v_i \sin \psi_i + \eta_y \\ -\frac{\alpha_g}{v_i} \tan \phi_i + \eta_\psi \\ f_\phi(\phi_i, \omega_i) \\ f_v(v_i, u_i) \end{Bmatrix} \quad (4)$$

where α_g denotes the gravitational acceleration, and $\eta_x, \eta_y, \eta_\psi$ represent the environmental disturbances in the x, y , and heading angle channels, respectively. The functions $f_\phi(\cdot)$ and $f_v(\cdot)$ describe the closed-loop roll and airspeed dynamics [31].

3) **Reward Function:** In the flocking with obstacle avoidance task, each follower aims to minimize its distance to the leader while avoiding collisions with neighboring AAVs and surrounding obstacles [24]. Accordingly, the reward function is designed as a weighted sum of multiple objectives

$$r_i = r_i^{\text{fol}} + \sum_{j=1}^{|\mathcal{N}_i|} r_{i,j}^{\text{col}} + \sum_{m=1}^9 r_{i,m}^{\text{obs}} \quad (5)$$

where

$$r_i^{\text{fol}} = \begin{cases} -P & \rho_{i,l} \leq R_s \\ \lambda_1 (1 - e^{p_1(R_a - \rho_{i,l})}) & R_s < \rho_{i,l} \leq R_a \\ \lambda_2 (1 - e^{p_2(\rho_{i,l} - R_a)}) & R_a < \rho_{i,l} \leq R_l \\ -K & \rho_{i,l} > R_l \end{cases} \quad (6)$$

$$r_{i,j}^{\text{col}} = \begin{cases} -P & \rho_{i,j} \leq R_s \\ \lambda_1 (1 - e^{p_1(R_a - \rho_{i,j})}) & R_s < \rho_{i,j} \leq R_a \\ 0 & \rho_{i,j} > R_a \end{cases} \quad (7)$$

$$r_{i,m}^{\text{obs}} = \begin{cases} -P & d_{i,m} \leq R_s \\ \lambda_1 (1 - e^{p_1(R_a - d_{i,m})}) & R_s < d_{i,m} \leq R_a \\ 0 & d_{i,m} > R_a \end{cases} \quad (8)$$

denote the *following reward*, *inter-AAV collision avoidance reward*, and *environmental obstacle avoidance reward*, respectively. Here, $\rho_{i,l}$ is the distance from the leader to follower i , R_l is the maximum communication range of the leader, R_a and R_s are the alert radius and minimum safety radius of followers, respectively. $\lambda_1 > 0$ and $\lambda_2 > 0$ are two weight coefficients; k_1 and k_2 are two small positive constants, while K is a large positive constant.

III. METHODOLOGY

To address the distributed flocking with obstacle avoidance problem, we propose a novel generative diffusion model enhanced multiagent actor-critic (GDMAAC) algorithm. As illustrated in Fig. 2, the GDMAAC algorithm comprises two main components: the diffusion-based policy network (actor) and the graph-based value network (critic). The actor is responsible for learning the individual policies for each agent (follower) by leveraging the GDM [26], while the critic is used to evaluate the joint actions of all agents by utilizing the GAT [32]. The policy and value networks operate under the centralized training with decentralized execution paradigm [33]. During training, global state from all agents is utilized to learn optimal policies and value functions. However, during execution, each agent acts independently based on its own local observations and its learned decentralized policy. In the following, we first introduce the diffusion-based policy network (see Section III-A) and the graph-based value network (see Section III-B), respectively. Afterward, we describe the learning process in detail (see Section III-C).

A. Diffusion-Based Policy Network

In order to enhance exploration and robustness in dynamic and uncertain environments, we design the diffusion-based policy network to generate optimal actions based on the observations of each agent. Specifically, the policy network utilizes a GDM to generate diverse actions through a denoising process, conditioned on the observations processed by a GAT. Below, we first introduce the GAT-based observation encoder and then the reverse denoising process in GDM.

1) **Observation Encoder:** In the policy network, the observation encoder is designed to process the observation of each agent. As indicated by (3), the length of the agent's observation is positively correlated with the number of its neighbors within the observation range. Since the number of neighbors is dynamic and varies over time, it is crucial to develop a network module that can handle these varying numbers of neighbors and convert them into a fixed-length feature vector. To achieve this, we design an observation encoder based on GAT, which allows for the dynamic processing of the agent's observation while ensuring

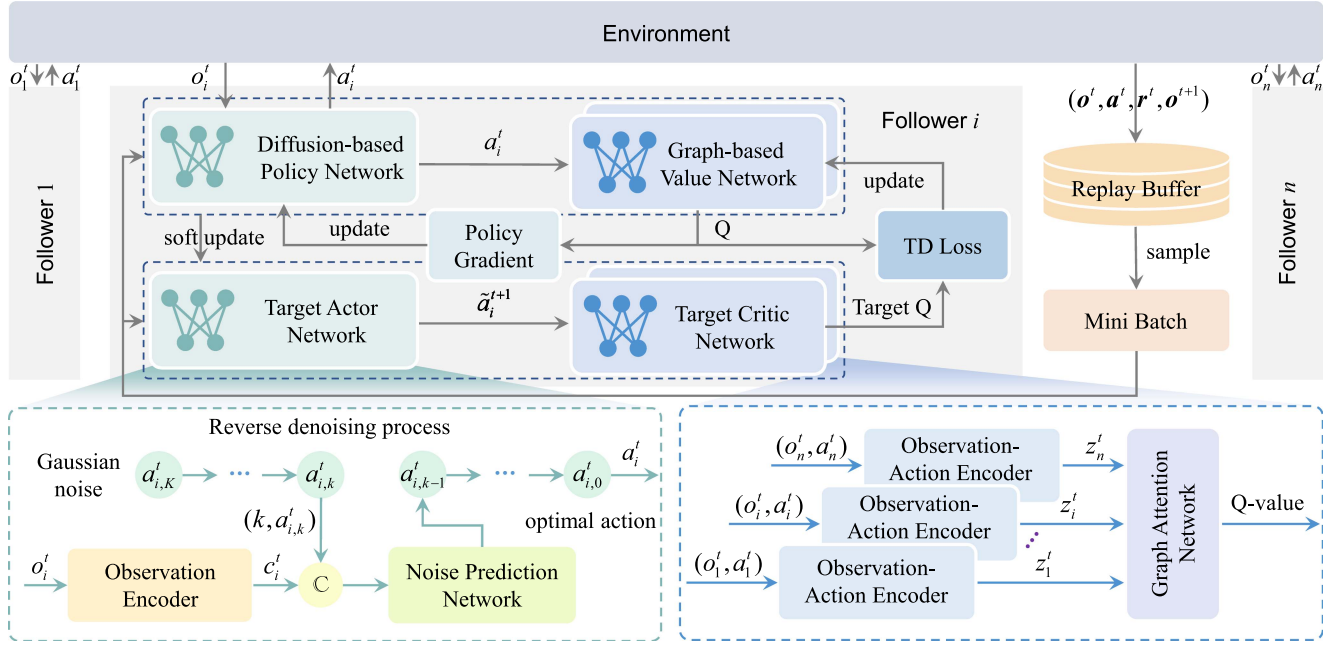


Fig. 2. Framework of GDMAAC. The actor leverages the GDM to generate diverse actions, while the critic utilizes the GAT to process and aggregate information from all agents. Here, the symbol $\mathbb{C}$ denotes the concatenation operation.

that the output embedding remains of a fixed size regardless of the number of neighbors.

We detail here the implementation process of the observation encoder. We first use four embedding layers to separately extract the features of the leader's state, ego state, neighbors' states, and the distance sensor's information

$$\begin{aligned} e_l^t &= \text{FC}(\xi_l^t), \quad e_i^t = \text{FC}(\xi_i^t) \\ e_j^t &= \text{FC}(\xi_j^t), \quad j \in \mathcal{N}_i^t \\ e_d^t &= \text{FC}(\text{Flatten}(\mathbf{d}_i^t)) \end{aligned} \quad (9)$$

where $\text{FC}(\cdot)$ represents a fully connected layer, $\text{Flatten}(\cdot)$ denotes the flatten operation.

We then aggregate the embedding of neighbors' states into a fixed-length vector through a GAT layer. Mathematically, the aggregation process can be formulated as

$$h_i^t = \sum_{j \in \mathcal{N}_i} \alpha_{ij}^t \mathbf{W}_{\text{nei}} e_j^t \quad (10)$$

where h_i^t represents the aggregated embedding of the neighbors' states for agent i at timestep t . $\mathbf{W}_{\text{nei}}$ is the learnable weight matrix that projects each e_j^t into a higher dimensional feature space, α_{ij}^t is the attention weight between agent i and its neighbor j , indicating the importance of j 's state in the aggregation process. Formally, the attention weight α_{ij}^t is computed by the following attention mechanism [32]:

$$\begin{aligned} \alpha_{ij}^t &= \frac{\exp(s_{ij}^t)}{\sum_{j \in \mathcal{N}_i} \exp(s_{ij}^t)} \\ s_{ij}^t &= \mathbf{a}^\top \text{LeakyReLU}([\mathbf{W}_{\text{ego}} e_i^t + \mathbf{W}_{\text{nei}} e_j^t]) \end{aligned} \quad (11)$$

where s_{ij}^t denote the similarity score; $\mathbf{a}$ is the learnable attention weight vector; $\mathbf{W}_{\text{ego}}$ is the learnable weight matrix that projects e_i^t into a higher dimensional feature space; $\text{LeakyReLU}(\cdot)$ represents the LeakyReLU activation function used to introduce nonlinearity.

After aggregating the neighbors' states, the final feature vector for agent i 's observation is obtained by concatenating the following embedding vectors:

$$c_i^t = e_l^t \parallel e_i^t \parallel h_i^t \parallel e_d^t \quad (12)$$

where $\parallel$ denotes the concatenation operation, c_i^t is the final aggregated feature vector for agent i , which is used as the conditional input for the diffusion model to generate actions.

2) Reverse Denoising Process: After encoding the agent's observation information, the policy network generates actions using a diffusion model, such as the denoising diffusion probabilistic model [26]. As a class of generative models, diffusion models have shown strong generative capabilities across various domains by progressively refining random noise into structured outputs. By leveraging these capabilities, the diffusion-based policy network can generate diverse and high-quality actions for complex multiagent coordination tasks.

In the proposed GDMAAC framework, only the reverse denoising process is adopted to serve as the policy generator. In conventional diffusion models, the forward process is a fixed, nonlearnable Markov chain used to progressively add Gaussian noise to a dataset of expert samples, providing supervision for learning the reverse process. However, in our online multiagent reinforcement learning setting, such expert data are unavailable, as optimal actions must be discovered through continuous interaction with the environment. Consequently, the forward process

is omitted [34], [35]. Instead, the reverse process is directly employed to iteratively refine Gaussian noise into optimal actions. This design allows the diffusion-based policy network to act as a generative policy model that learns through trial-and-error optimization rather than supervised reconstruction.

Formally, the reverse denoising process aims to generate the optimal action $a_{i,0}^t$ by iteratively refining the Gaussian noise $a_{i,K}^t \sim \mathbb{N}(0, I)$ through K denoising step, conditional on the observation encoder's output c_i^t . Specifically, this process is modeled as follows [26]:

$$p_\theta(a_{i,k-1}^t | a_{i,k}^t) = \mathbb{N}(a_{i,k-1}^t; \mu_\theta(a_{i,k}^t, k, c_i^t), \tilde{\beta}_t I) \quad (13)$$

where $k = K, \dots, 1$ represents the reverse diffusion timestep (distinct from the trajectory timestep, which is indicated by the subscript t), $p_\theta(a_{i,k-1}^t | a_{i,k}^t)$ denotes a Gaussian distribution defining the transition from $a_{i,k}^t$ to $a_{i,k-1}^t$. The adjusted variance of the distribution $p_\theta(a_{i,k-1}^t | a_{i,k}^t)$ is calculated by

$$\tilde{\beta}_k = \frac{1 - \bar{\alpha}_{k-1}}{1 - \bar{\alpha}_k} \beta_k \quad (14)$$

where $\beta_k \in (0, 1)$ is a predetermined variance factor which controls the noise scale at timestep k , $\alpha_k = 1 - \beta_k$, $\bar{\alpha}_k = \prod_{s=1}^k \alpha_s$. The mean $\mu_\theta(a_{i,k}^t, k, c_i^t)$ of the reverse transition distribution can be constructed leveraging a noise prediction network

$$\mu_\theta(a_{i,k}^t, k, c_i^t) = \frac{1}{\sqrt{\alpha_k}} \left(a_{i,k}^t - \frac{\beta_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta(a_{i,k}^t, k, c_i^t) \right). \quad (15)$$

The noise prediction network ϵ_θ approximates the noise component conditioned on c_i^t

$$\epsilon_\theta(a_{i,k}^t, k, c_i^t) = \text{MLP}_\epsilon(a_{i,k}^t \parallel \text{PE}(k) \parallel c_i^t) \quad (16)$$

where $\text{MLP}(\cdot)$ denotes a multilayer perceptron (MLP), $\text{PE}(k)$ denotes the sinusoidal positional encoding of timestep k .

Finally, we can obtain an action by drawing samples from the learned reverse transition distribution with the reparameterization trick

$$a_{i,k-1}^t = \mu_\theta(a_{i,k}^t, k, c_i^t) + \sqrt{\tilde{\beta}_t} \epsilon \quad (17)$$

where $\epsilon \sim \mathbb{N}(0, I)$ is a noise term drawn from a normal distribution. By iterating through this sample process, the agent can generate the optimal action (denoised action $a_{i,0}^t$).

B. Graph-Based Value Network

To evaluate the actions of all agents and promote cooperative behavior, we construct a centralized value network based on GAT. The graph-based value network can effectively capture the relational dependencies among agents by modeling them as nodes in an interaction graph. Each node encodes its own observation and action information, and the GAT module aggregates messages from neighbors to compute the final Q-value estimation. In this section, we detail the design of the value network, which comprises two modules: the observation-action encoder and Q-value estimation with GAT.

1) Observation-Action Encoder: The input to the value network consists of each agent's local observation and its executed action. To construct a meaningful representation that captures both the state and action information, we design an encoder that processes these inputs and maps them into a fixed-dimensional embedding.

For each agent i at timestep t , we first encode the observation o_i^t and the action a_i^t independently. As discussed in Section III-A1, the observation o_i^t has variable length. Thus, the same GAT-based observation encoder introduced in Section III-A1 is used to transform o_i^t into a fixed-size embedding. The action embedding can be easily obtained through an FC layer. Formally, the observation embedding c_i^t and the action embedding b_i^t can be expressed as follows:

$$c_i^t = \text{OE}(o_i^t), \quad b_i^t = \text{FC}(a_i^t) \quad (18)$$

where $\text{OE}(\cdot)$ denotes the observation encoder defined in Section III-A1. The outputs of the observation encoder and the action encoder are then concatenated and passed through an FC layer to obtain the initial feature representation

$$z_i^t = \text{FC}(c_i^t \parallel b_i^t). \quad (19)$$

This representation z_i^t serves as the input node feature in the subsequent GAT, which enables the agent to aggregate contextual information from its neighbors in the multiagent system.

2) Q-Value Estimation With GAT: When estimating the state-action value (Q-value) for each agent, it is beneficial to selectively focus on the most relevant information from other agents rather than treating all agents equally [36]. In light of this, we utilize a GAT layer to capture the influence of other agents on the agent's Q-value.

Similar to the observation encoder for agent i , the feature aggregation process is computed based on the importance of all agents

$$v_i^t = \sum_{j \in \mathcal{I}/i} \beta_{ij}^t \mathbf{W}_r z_j^t \quad (20)$$

where the importance coefficient β_{ij}^t is calculated as

$$\beta_{ij}^t = \frac{\exp(\mathbf{b}^\top \text{LeakyReLU}(\mathbf{W}_l z_i^t + \mathbf{W}_r z_j^t))}{\sum_{j \in \mathcal{I}/i} \exp(\mathbf{b}^\top \text{LeakyReLU}(\mathbf{W}_l z_i^t + \mathbf{W}_r z_j^t))} \quad (21)$$

where $\mathbf{b}$ is a learnable attention vector, $\mathbf{W}_l$ and $\mathbf{W}_r$ are learnable weight matrices, $j \in \mathcal{I}/i$ means $j \in \mathcal{I}$ but $j \neq i$.

This selective aggregation allows the agent to focus on the most important neighboring agents for estimating its Q-value. Formally, the aggregated feature v_i^t is combined with agent i 's own feature z_i^t to approximate the final Q-value through a three-layer MLP

$$Q_i^t = \text{MLP}_Q(z_i^t \parallel v_i^t). \quad (22)$$

C. Learning Process

In this section, we present the training procedure of the proposed GDMAAC algorithm for solving the distributed flocking with obstacle avoidance task. Unlike conventional multiagent actor-critic approaches that learn independent policies for each

agent [33], GDMAAC adopts the parameter sharing mechanism [37] to exploit the homogeneity among agents, thereby improving scalability and sample efficiency. As a result, only one shared diffusion-based actor and one centralized GAT-based critic are trained for all agents.

To train the actor and critic networks, we maintain a shared replay buffer $\mathcal{D}$, which stores the experiences collected from all agents. At each training iteration, a minibatch of transitions is randomly sampled from $\mathcal{D}$ to update the networks. The GAT-based critic network is updated by minimizing the temporal difference loss [30], [33]

$$\mathcal{L}(\theta_l^Q) = \mathbb{E}_{(\mathbf{o}^t, \mathbf{a}^t, \mathbf{r}^t, \mathbf{o}^{t+1}) \sim \mathcal{D}} \left[\frac{1}{n} \sum_{i=1}^n (Q_l(\mathbf{o}^t, \mathbf{a}^t) - y_i^t)^2 \right] \quad (23)$$

where $\theta_{l=1,2}^Q$ denotes the parameters of the critic network; y_i^t is the target Q-value; $\mathbf{o}^t = (o_1^t, \dots, o_n^t)$, $\mathbf{a}^t = (a_1^t, \dots, a_n^t)$, and $\mathbf{r}^t = (r_1^t, \dots, r_n^t)$ represents the joint observations, actions, and rewards for all agents at timestep t , respectively.

To mitigate overestimation bias in computing the target Q-value, we incorporate the double Q-learning technique [38]. Specifically, two independent critic networks Q_1 and Q_2 are trained, and the minimum of them is used to calculate the target Q-value

$$y_i^t = r_i^t + \gamma \min_{l=1,2} Q_l^-(\mathbf{o}^{t+1}, \tilde{a}_1^{t+1}, \dots, \tilde{a}_n^{t+1}) \quad (24)$$

where Q_l^- denotes the target critic network, $\gamma \in [0, 1)$ is the discounted factor, and $\tilde{a}_i^{t+1}$ indicates the action generated from the target actor network with smoothing noise [38]

$$\tilde{a}_i^{t+1} = \pi^-(o_i^{t+1}) + \epsilon_a, \quad \epsilon_a \sim \text{clip}(\mathcal{N}(0, \sigma_a^2), -c, c) \quad (25)$$

where π^- denotes the target actor network, σ_a^2 indicates the variance of Gaussian noise ϵ_a , and $\text{clip}(\cdot, -c, c)$ represents the clip operation, which limits values to the range $[-c, c]$.

The diffusion-based policy network is optimized to maximize the expected Q-value, with gradients computed via backpropagation through the reverse denoising steps

$$\begin{aligned} \nabla_{\theta^\pi} \mathcal{J}(\theta^\pi) = & \mathbb{E}_{(\mathbf{o}^t, \mathbf{a}^t, \mathbf{r}^t, \mathbf{o}^{t+1}) \sim \mathcal{D}} \left[\frac{1}{n} \sum_{i=1}^n (\nabla_{\theta^\pi} \pi(o_i^t) \right. \\ & \left. \times \nabla_{a_i^t} Q_1(\mathbf{o}^t, a_1^t, \dots, a_n^t) \mid_{a_i^t = \pi(o_i^t)} \right]. \end{aligned} \quad (26)$$

To stabilize the learning process, we employ the target actor network π^- and target critic networks $Q_{l=1,2}^-$ in GDMAAC. Their parameters θ^{π^-} and $\theta_{l=1,2}^{Q^-}$ are updated as follows:

$$\begin{aligned} \theta^{\pi^-} & \leftarrow \tau \theta^\pi + (1 - \tau) \theta^{\pi^-} \\ \theta_l^{Q^-} & \leftarrow \tau \theta_l^Q + (1 - \tau) \theta_l^{Q^-} \end{aligned} \quad (27)$$

where $\tau \ll 1$ is the soft update rate.

The complete learning pipeline of the proposed GDMAAC algorithm is summarized in Algorithm 1.

Algorithm 1: GDMAAC.

Input: Maximum training episodes M , maximum timestep per episode N_s , update frequency C , target update rate τ , replay buffer maximum capacity N , batch size N_b , noise parameters σ_a^2, c

- 1: Initialize diffusion-based actor with parameters θ^μ and two graph-based critics with parameters $\theta_{l=1,2}^Q$ randomly
- 2: Initialize target networks: $\theta^{\pi^-} \leftarrow \theta^\pi$; $\theta_{l=1,2}^{Q^-} \leftarrow \theta_{l=1,2}^Q$
- 3: Initialize replay buffer $\mathcal{D}$ with maximum capacity N
- 4: **for** episode = 1 to M **do**
- 5: Receive initial observations $\mathbf{o}^1$
- 6: **for** $t = 1$ to N_s **do**
- 7: Generate action a_i^t for each agent with reverse denoising process
- 8: Execute joint actions $\mathbf{a}^t = (a_1^t, \dots, a_n^t)$, compute immediate reward r_i^t , and receive new observations $\mathbf{o}^{t+1}$
- 9: Store transition $(\mathbf{o}^t, \mathbf{a}^t, \mathbf{r}^t, \mathbf{o}^{t+1})$ in $\mathcal{D}$, delete the oldest transition if $\|\mathcal{D}\| > N$
- 10: $\mathbf{o}^t \leftarrow \mathbf{o}^{t+1}$
- 11: Sample a minibatch of N_b tuples from $\mathcal{D}$
- 12: Update $\theta_{l=1,2}^Q$ by minimizing $\mathcal{L}(\theta_l^Q)$ in (23)
- 13: **if** $t \bmod C$ **then**
- 14: Update θ^π via gradient in (26)
- 15: Update θ^{π^-} and $\theta_{l=1,2}^{Q^-}$ with (27)
- 16: **end if**
- 17: **end for**
- 18: **end for**

IV. SIMULATION AND EXPERIMENT

This section validates the proposed approach in numerical simulations and HITL experiments.

A. Implementation Details

We now detail the parameter settings of the proposed GDMAAC algorithm. For the diffusion-based policy network, the four FC layers in (9) for embedding all contained 64 neurons. When computing the attention weight in (11), $\mathbf{W}_{\text{nei}} \in \mathbb{R}^{64 \times 64}$, $\mathbf{W}_{\text{ego}} \in \mathbb{R}^{64 \times 64}$, $\mathbf{a} \in \mathbb{R}^{64 \times 1}$. The MLP in the noise prediction network (16) contained three FC hidden layers, each of which had 256 neurons. For the graph-based value network, the FC layers in (18) and (19) had 64 and 256 neurons, respectively. In (21), $\mathbf{W}_l \in \mathbb{R}^{256 \times 256}$, $\mathbf{W}_r \in \mathbb{R}^{256 \times 256}$, and $\mathbf{b} \in \mathbb{R}^{256 \times 1}$. The two hidden layers of the three-layer MLP in (22) for estimating the Q-value both had 256 neurons, but the output layers had one neuron activated by the linear functions. All other FC layers were followed by the Mish activation function [39].

In the diffusion-based policy network, the reverse step was set to $K = 5$. The predetermined variance factor $\beta_k = 1 - e^{-\frac{\beta_{\min}}{K} - \frac{2k-1}{2K^2}(\beta_{\max} - \beta_{\min})}$ was controlled by the variational posterior scheduler [26], where $\beta_{\min}$ and $\beta_{\max}$ were set to 0.1 and 10, respectively. The output of the diffusion-based policy was clipped into the interval $[-1, 1]$. We used the Adam optimizer

TABLE I
PARAMETER SETTINGS

Name	Value	Name	Value
R_l	250 m	R_c	100 m
R_a	50 m	R_s	5 m
$\omega_{\max}$	10°	$u_{\max}$	1 m/s
λ_1	10	λ_2	100
p_1	0.1	p_2	0.01
P	1000	K	5
N_b	32	N_s	60
N	100000	γ	0.95
τ	0.005	C	2
σ_a^2	0.2	c	0.5

to update all network parameters of GDMAAC. The learning rates for updating the policy and the value network were set to 0.0001 and 0.0005, respectively [33]. The maximum training episode was set to $M = 10\,000$. In general, the parameter settings were set with reference to empirical tests and previous works [24], [26], [33]. The empirical values of other parameters in GDMAAC are presented in Table I.

B. Numerical Simulation Results

1) *Training Results*: We first compared our algorithm with several existing state-of-the-art baselines, i.e., MADDPG [33], AMADDPG, ARMADDPG, HRAMAAC, and HRAMATD3. To enable scalability of the learned flocking policies to different AAV swarm sizes, the five baseline algorithms also adopted the parameter sharing mechanism [37] as in GDMAAC, and utilized the self-attention mechanism [40] as the observation encoder following [30]. The five baselines also adopted the parameter sharing mechanism [37], consistent with GDMAAC, and utilized self-attention [40] as the observation encoder, following the approach in [30]. In addition, AMADDPG applied the self-attention mechanism in the critic to selectively aggregate information from other agents [36]. ARMADDPG further introduced the recurrent actor-critic framework, such as RMADDPG [41]. Compared with ARMADDPG, HRAMAAC incorporated the group-level attention mechanism consistent with [24]. On this basis, HRAMATD3 further enhanced the learning process with the MATD3 algorithm [42].

We trained the five baseline algorithms and our GDMAAC algorithm independently to acquire flocking policies. In the training scenario, ten follower AAVs navigated alongside a leader within a rectangular area measuring $1200\text{ m} \times 200\text{ m}$. The scenario included twenty randomly placed obstacles, either convex polygons or circles. For each training episode, the obstacle configurations and the leader's planned trajectory were randomly generated. The radius of the obstacles were randomly generated from the predefined set $\{10, 15, 20, 25, 30, 35, 40\}$ (m). This configuration ensures that obstacles are large enough to be detected by the AAV's distance sensor.

Fig. 3 presents the average rewards obtained by the followers with the five baselines and our GDMAAC algorithm. As observed, all learning curves exhibit a steep increase during the early training phase. After approximately 4000 episodes,

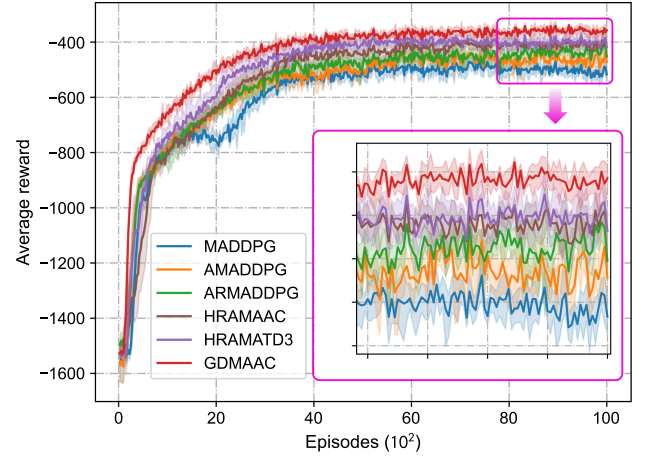


Fig. 3. Learning curves of MADDPG, AMADDPG, ARMADDPG, HRAMAAC, HRAMATD3, and GDMAAC. Note that the reward values are averaged every 25 episodes.

all of them begin to stabilize and reach convergence (about 3.5 h of wall-clock time with NVIDIA GeForce RTX 4090D GPU). Although the learning efficiency is similar, the final convergence values vary significantly among the algorithms. The final rewards of AMADDPG and ARMADDPG are higher than that of MADDPG. The explanation for performance improvement is that selecting relevant information from other agents and encoding historical information are very useful. Compared with ARMADDPG, HRAMAAC and HRAMATD3 achieve a higher average reward. This means that it is more effective to use group-level attention to discriminatively aggregate embeddings of different groups than to concatenate them directly. In addition, GDMAAC achieves a higher average reward compared to HRAMAAC and HRAMATD3. The above-mentioned results demonstrate that the proposed GDMAAC algorithm outperforms all five existing baselines.

2) *Testing Results*: After the training phase, we tested the flocking control with obstacle avoidance policy acquired by the proposed GDMAAC algorithm in a representative task scenario. Thirty-six convex polygonal and circular obstacles were evenly distributed across the rectangular mission area. The leader followed a predefined path without considering environmental obstacles, aiming to evaluate whether the followers would collide with the obstacles as the leader passed through them. Meanwhile, ten follower AAVs executed the learned GDMAAC policy to track the leader over 180 timesteps.

The trajectories and performance of the followers are depicted in Figs. 4 and 5, respectively. The followers can keep up with the leader and keep away from the surrounding obstacles at the same time, whether the leader flies in a straight line or in violent motion. More precisely, the distances between the leader and the followers (ρ) are predominantly maintained within the range of 50–150 meters during the evaluation. Furthermore, both the minimum interfollower distance ($D_{\min}^{\text{nei}}$) and the minimum distance between the followers and surrounding obstacles ($D_{\min}^{\text{oba}}$) consistently exceed the follower safety threshold $R_s = 5\text{ m}$ at all timesteps. These results indicate that no collisions occur

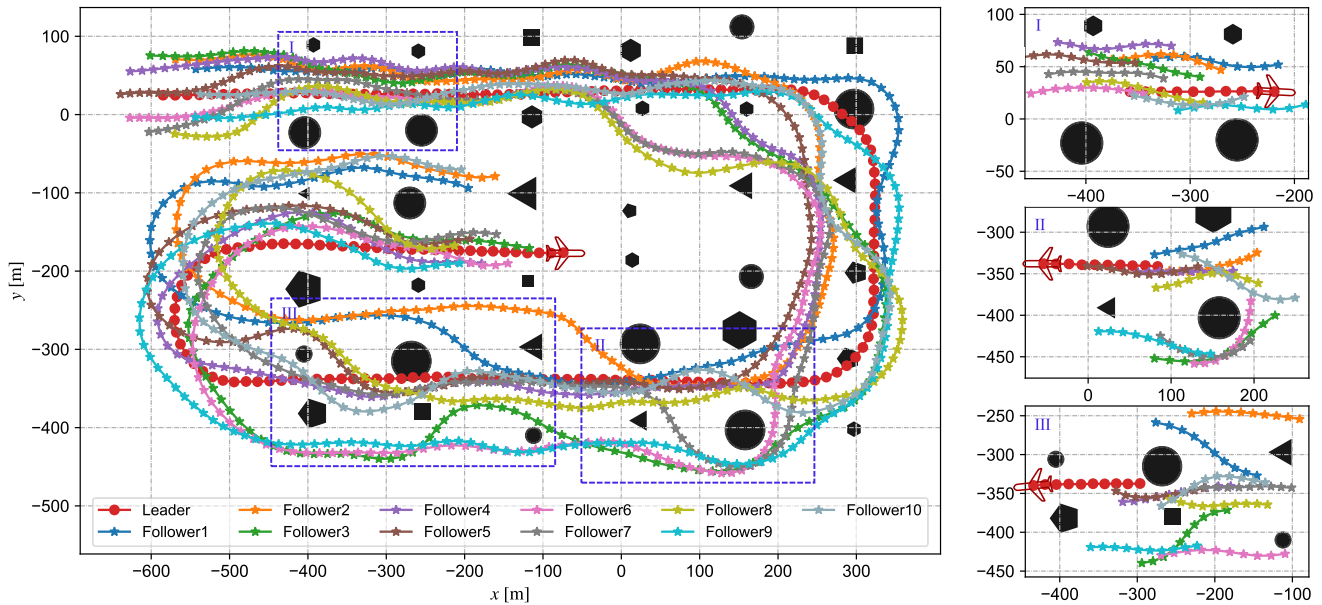


Fig. 4. Trajectories of ten followers employing the learned GDMAAC policy in a typical obstacle-cluttered environment.

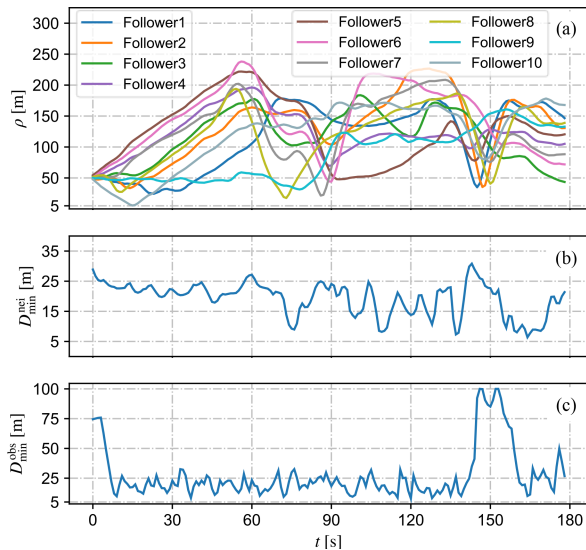


Fig. 5. Performance of the learned GDMAAC policy. (a) Distances from the leader to the followers. (b) Minimum interfollower distance. (c) Minimum distance between followers and surrounding obstacles.

among followers or between any follower and environmental obstacles throughout the entire testing process. In addition, we also conducted two testing experiments with five and 20 followers under the same environmental settings. Because of the limitation of length, the testing results are available in the supplemental movie at.¹

3) Generalization Results: To evaluate the generalization and scalability of the learned flocking control with obstacle

avoidance policies, we directly applied it to AAV swarms comprising varying numbers of follower agents. Specifically, generalization evaluations were conducted with swarm sizes of ten, 15, and 20 followers. Each evaluation consisted of 20 episodes, with each episode running for 60 timesteps. We defined the following metrics to evaluate the policy performance: the average reward of each follower at each timestep G , the average collision rate F , and the average distance between the leader and the followers $\bar{\rho}$.

Table II reports the zero-shot generalization results on the AAV swarms with different numbers of followers. It is not surprising that MADDPG achieves the worst performance across three scenarios. Compared with MADDPG, AMADDPG and ARMADDPG perform better in terms of the three metrics. We note that HRMAAC and HRMATD3 achieve better and similar performance. Although the collision rate of HRMAAC is lower than that of HRMATD3, HRMATD3 performs better in terms of the average distance to the leader and the average reward. In contrast, our proposed GDMAAC algorithm significantly outperforms all baselines in terms of average reward and collision rate. More importantly, GDMAAC shows a significant advantage in reducing collision rate. Specifically, GDMAAC reduces the collision rate from 1.85% to 1.42% at $n = 10$, representing a relative reduction of 23.24%. Similarly, at $n = 20$, the collision rate decreases from 2.38% to 1.72%, corresponding to a 27.73% reduction. These results indicate that GDMAAC produces safer and more reliable policies, especially in dense swarm scenarios.

It is worth noting that the generalization simulations were conducted using the same trained GDMAAC model, without any retraining for different swarm sizes. This scalability stems from the parameter sharing mechanism, which enables the learned policy to generalize directly to swarms of arbitrary sizes, and from the GAT, which allows the policy and value networks

¹[Online]. Available: <https://youtu.be/OfCYR3KYc2Q>

TABLE II
GENERALIZATION RESULTS OF THE LEARNED FLOCKING POLICIES ON THE AAV SWARMS WITH DIFFERENT NUMBERS OF FOLLOWERS

Method	$n = 10$			$n = 15$			$n = 20$		
	G	F (%)	$\bar{\rho}$ (m)	G	F (%)	$\bar{\rho}$ (m)	G	F (%)	$\bar{\rho}$ (m)
MADDPG	-401.97 ± 9.00	3.41 ± 0.36	87.78 ± 6.04	-502.32 ± 22.88	3.94 ± 0.48	87.25 ± 4.85	-609.69 ± 23.24	4.90 ± 0.28	87.72 ± 4.77
AMADDPG	-381.92 ± 42.46	2.71 ± 1.10	88.63 ± 13.57	-479.65 ± 52.79	3.07 ± 1.03	90.89 ± 14.02	-604.83 ± 74.94	3.99 ± 1.17	91.70 ± 14.95
ARMADDPG	-324.51 ± 16.51	2.39 ± 0.68	87.72 ± 7.36	-418.48 ± 38.19	2.95 ± 0.98	87.45 ± 7.13	-530.42 ± 45.09	4.04 ± 1.20	87.77 ± 6.73
HRMAAC	-313.49 ± 20.19	1.43 ± 0.16	100.56 ± 9.05	-376.19 ± 16.39	1.82 ± 0.10	101.56 ± 4.68	-445.16 ± 15.24	2.06 ± 0.33	102.17 ± 4.72
HRMATD3	-310.57 ± 16.76	1.85 ± 0.24	92.94 ± 3.67	-372.92 ± 16.58	1.93 ± 0.17	95.22 ± 1.89	-439.79 ± 10.48	2.38 ± 0.15	97.55 ± 2.48
GDMAAC	-279.00 ± 19.22	1.42 ± 0.23	95.16 ± 7.13	-332.76 ± 20.48	1.45 ± 0.25	97.61 ± 6.35	-421.75 ± 25.45	1.72 ± 0.18	98.49 ± 8.88

Bold values indicate the best performance.

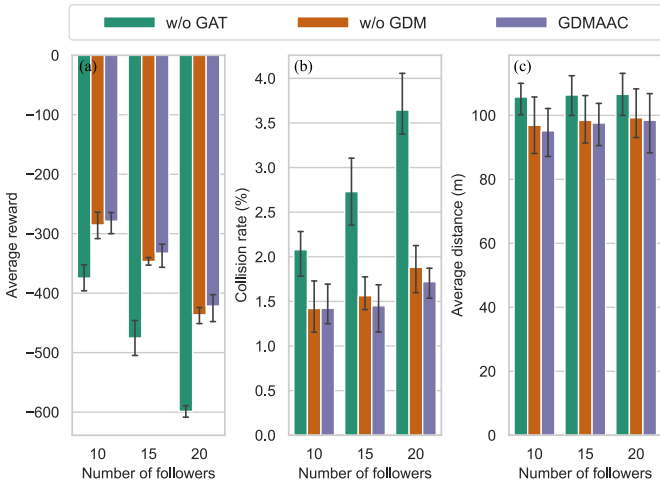


Fig. 6. Bar charts of ablation results of the proposed algorithms for flocking control with different numbers of followers. (a) Average reward. (b) Collision rate. (c) Average distance.

to process information from a variable number of neighbors. Overall, we can conclude from Table II that regardless of the size of AAV fleets, the proposed GDMAAC algorithm outperforms the four baselines and achieves the best tradeoff between the average reward F and the average distance $\bar{\rho}$.

4) Ablation Studies: To validate the effectiveness of each key component in the proposed GDMAAC algorithm, we conducted ablation studies by systematically removing the GDM and the GAT from the policy and value networks, respectively. Specifically, we designed the following two ablation variants. 1) w/o GDM: The diffusion-based policy network is replaced with a deterministic policy network as in the standard MADDPG algorithm. 2) w/o GAT: The graph attention mechanism is removed from the Q-value estimator. Instead, a simple concatenation operation is adopted for feature aggregation. The simulation settings of the ablation studies were the same as in Section IV-B3.

Fig. 6 illustrates the performance of the ablation variants across different swarm sizes. As shown, removing the GAT module (w/o GAT) results in significant performance degradation. For example, when $n = 15$, the average reward G decreases from -332.76 to -475.42, corresponding to a relative drop of 42.81%. Meanwhile, the collision rate F increases from

1.45% to 2.73%, reflecting an 88.28% rise. These results highlight the critical role of the graph-based value network in effectively capturing inter-agent interactions and enhancing policy evaluation accuracy. On the other hand, removing the GDM module (w/o GDM) also leads to noticeable performance deterioration, though its impact is less severe compared to the removal of GAT. For instance, when $n = 20$, the average reward G declines from -421.75 to -436.46, and the collision rate increases from 1.72% to 1.88%, indicating a relative reward decrease of 3.49% and a collision rate increase of 9.30%. This confirms that GDM contributes to improving policy diversity and robustness. Overall, GDMAAC consistently achieves the best performance across all metrics and scenarios. The ablation studies demonstrate that both the diffusion-based policy and the graph-based value network are indispensable components, jointly contributing to superior flocking stability, lower collision rates, and better scalability of the proposed approach.

C. HITL Experiment Results

To validate the practicability of the proposed approach for flocking control with obstacle avoidance, we built a high-fidelity HITL experiment system [30], [43]. As illustrated in Fig. 7, the X-Plane ten flight simulator provided a realistic virtual environment for testing the control and coordination of fixed-wing AAVs. Each onboard Jetson TX2 computer, running Ubuntu 18.04 LTS and robot operating system, was responsible for executing the learned policy and generating high-level control commands in real time. These commands were transmitted to the PX4/Pixhawk autopilot for low-level flight control. The ground control station was used for monitoring and centralized management of the entire AAV swarm.

In the HITL experiment, four follower AAVs deployed the flocking policy learned from numerical simulations without any parameter tuning to follow a predefined leader. The task area contained nine circular obstacles of varying radii, as illustrated in Fig. 8. The leader's trajectory was preplanned based on the positions of the destination and surrounding obstacles. The entire evaluation was conducted over a duration of 180 timesteps (i.e., seconds). Fig. 8 presents the trajectories of the leader and the four followers in this HITL experiment. The four followers can autonomously keep up with the leader while successfully avoiding environmental obstacles, even in a narrow corridor

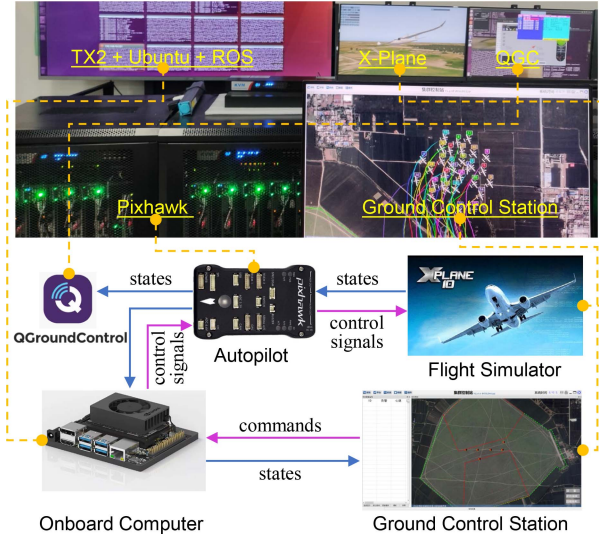


Fig. 7. High-fidelity HITL experiment system for flocking control with fixed-wing AAV swarms.

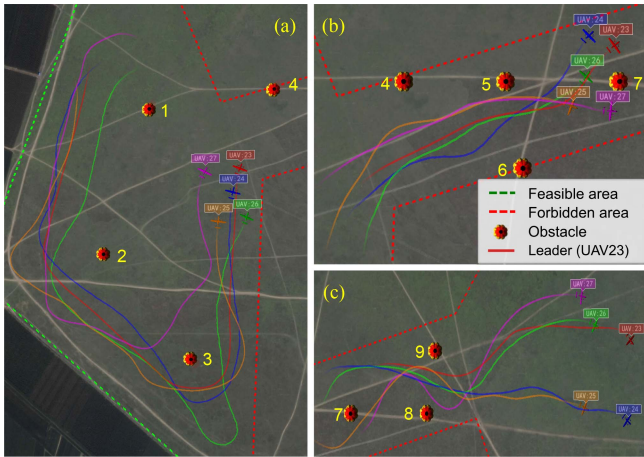


Fig. 8. Trajectories of four followers and the leader in the HITL experiment. (a) $t = 0 \sim 110$ s. (b) $t = 110 \sim 130$ s. (c) $t = 130 \sim 180$ s.

with dense obstacles [see Fig. 8(b)-II]. A video about this HITL experiment is available at.²

The performance metrics of the learned GDMAAC flocking policy in this HITL experiment were also recorded. As illustrated in Fig. 9(a), the distances between the leader and the followers (ρ) remain within the range of 5–200 m, indicating stable leader-following behavior and effective avoidance of collisions with the leader. More importantly, as shown in Fig. 9(b) and (c), both the minimum interfollower distance ($D_{\min}^{\text{nei}}$) and the minimum distance between followers and nearby obstacles ($D_{\min}^{\text{obs}}$) consistently exceeded the predefined safety threshold $R_s = 5$ m. These results confirm that the learned GDMAAC policy enables each follower to successfully avoid collisions with both teammates and environmental obstacles while maintaining a flock.

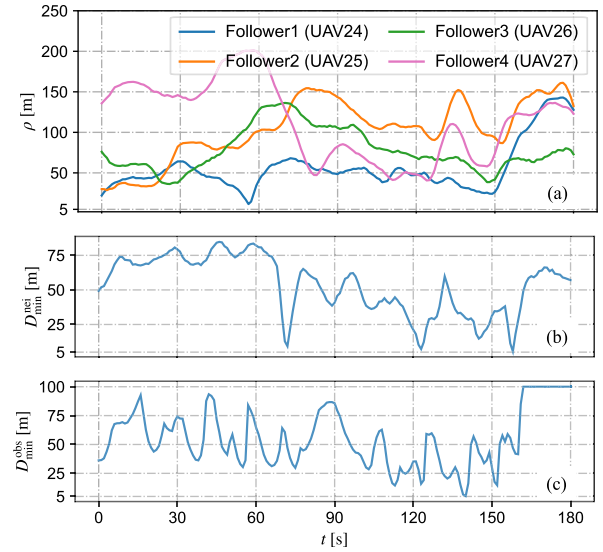


Fig. 9. Performance of the learned GDMAAC policy in the HITL experiment. (a) Distances to the leader. (b) Minimum interfollower distance. (c) Minimum distance between the followers and surrounding obstacles.

V. CONCLUSION

In this article, we propose the GDMAAC algorithm to address the distributed flocking control and obstacle avoidance problem for scalable fixed-wing AAV swarms. By integrating a GDM into the policy network, GDMAAC enhances agents' ability to generate diverse and multimodal actions. In addition, a GAT-based centralized value network is introduced to improve the accuracy of Q-value estimation by extracting relevant interagent features. These designs enable each AAV to explore multiple feasible maneuver strategies for flocking and obstacle avoidance, enhancing the robustness and flexibility of swarm coordination. Numerical simulations and HITL experiments demonstrate that GDMAAC significantly outperforms existing baselines in terms of flocking stability and collision avoidance, while maintaining strong generalization to varying swarm sizes. In future work, we plan to extend the GDMAAC framework to tackle more challenging scenarios, such as 3-D dynamic environments and heterogeneous AAV swarms.

REFERENCES

- [1] Y. Wang, Y. Yue, M. Shan, L. He, and D. Wang, "Formation reconstruction and trajectory replanning for multi-UAV patrol," *IEEE/ASME Trans. Mechatron.*, vol. 26, no. 2, pp. 719–729, Apr. 2021.
- [2] X. Zhang, H. Zhao, J. Wei, C. Yan, J. Xiong, and X. Liu, "Cooperative trajectory design of multiple UAV base stations with heterogeneous graph neural networks," *IEEE Trans. Wireless Commun.*, vol. 22, no. 3, pp. 1495–1509, Mar. 2023.
- [3] L. Zewde and I. A. Raptis, "Conceptualizing UAM: Technologies and methods for safe and efficient urban air transportation," *Green Energy Intell. Transp.*, vol. 5, no. 1, 2026, Art. no. 100265.
- [4] M. Zhang, C. Yan, W. Dai, X. Xiang, and K. H. Low, "Tactical conflict resolution in urban airspace for unmanned aerial vehicles operations using attention-based deep reinforcement learning," *Green Energy Intell. Transp.*, vol. 2, no. 4, 2023, Art. no. 100107.
- [5] S. Deniz, Y. Wu, Y. Shi, and Z. Wang, "A reinforcement learning approach to vehicle coordination for structured advanced air mobility," *Green Energy Intell. Transp.*, vol. 3, no. 2, 2024, Art. no. 100157.

²[Online]. Available: <https://youtu.be/OfCYR3KYc2Q>

- [6] Y. Deng, B. Zhu, and H. Duan, "Bioinspired bearing-based target enclosing control for unmanned aerial vehicle swarm," *IEEE/ASME Trans. Mechatron.*, vol. 30, no. 4, pp. 2699–2709, Aug. 2025, doi: [10.1109/TMECH.2024.3457826](https://doi.org/10.1109/TMECH.2024.3457826).
- [7] P.-X. Wu, C.-C. Yang, and T.-H. Cheng, "Cooperative transportation of UAVs without inter-UAV communication," *IEEE/ASME Trans. Mechatron.*, vol. 28, no. 4, pp. 2340–2351, Aug. 2023.
- [8] S. Dey and H. Xu, "Distributed adaptive flocking control for large-scale multiagent systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 2, pp. 3126–3135, Feb. 2025.
- [9] G. Zhu, Q. Zhang, B. Zhu, and T. Hu, "Heuristic predictive control for multirobot flocking in congested environments," *IEEE/ASME Trans. Mechatron.*, vol. 30, no. 2, pp. 1435–1446, Apr. 2025.
- [10] F. Wang, J. Huang, K. H. Low, and T. Hu, "Collective navigation of aerial vehicle swarms: A flocking inspired approach," *IEEE Trans. Intell. Veh.*, vol. 9, no. 1, pp. 1040–1053, Jan. 2024.
- [11] C. Li et al., "Flocking navigation and obstacle avoidance for UAV swarms via adaptive risk avoidance willingness mechanism," *IET Cyber- Syst. Robot.*, vol. 7, no. 1, 2025, Art. no. e70009.
- [12] S. Shao, J. Zhang, T. Wang, A. Shankar, and C. Maple, "Dynamic obstacle-avoidance algorithm for multi-robot flocking based on improved artificial potential field," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 4388–4399, Feb. 2024.
- [13] T. T. Nguyen, N. D. Nguyen, and S. Nahavandi, "Deep reinforcement learning for multiagent systems: A review of challenges, solutions, and applications," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3826–3839, Sep. 2020.
- [14] W. Du and S. Ding, "A survey on multi-agent deep reinforcement learning: From the perspective of challenges and applications," *Artif. Intell. Rev.*, vol. 54, no. 5, pp. 3215–3238, 2021.
- [15] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robot. Res.*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [16] J. Shi et al., "Multi-UAVs flocking for navigation and obstacle avoidance in network-constrained environments," *IEEE Internet Things J.*, vol. 12, no. 7, pp. 8931–8946, Apr. 2025.
- [17] J. Xiao, C. Huang, G. Yuan, Y. Wang, H. Jia, and Z. Wang, "A model learning based multiagent flocking collaborative control method for stochastic communication environment," *IEEE Trans. Ind. Informat.*, vol. 20, no. 6, pp. 8896–8906, Jun. 2024.
- [18] Y. Jia et al., "A deep reinforcement learning approach using asymmetric self-play for robust multirobot flocking," *IEEE Trans. Ind. Informat.*, vol. 21, no. 4, pp. 3266–3275, Apr. 2025.
- [19] S.-M. Hung and S. N. Givigi, "A Q-learning approach to flocking with UAVs in a stochastic environment," *IEEE Trans. Cybern.*, vol. 47, no. 1, pp. 186–197, Jan. 2017.
- [20] Y. Song et al., "Distributed swarm system with hybrid-flocking control for small fixed-wing UAVs: Algorithms and flight experiments," *Expert Syst. with Appl.*, vol. 229, 2023, Art. no. 120457.
- [21] H. Li, Z. Sun, H. Chen, X. Wang, and L. Shen, "Bearing-based simultaneous localization and affine formation tracking for fixed-wing unmanned aerial vehicles," *Aerosp. Sci. Technol.*, vol. 157, 2025, Art. no. 109840.
- [22] B. Ma et al., "Reinforcement learning based UAV formation control in GPS-denied environment," *Chin. J. Aeronaut.*, vol. 36, no. 11, pp. 281–296, 2023.
- [23] X. Zhuang, D. Li, Y. Wang, X. Liu, and H. Li, "Optimization of high-speed fixed-wing UAV penetration strategy based on deep reinforcement learning," *Aerosp. Sci. Technol.*, vol. 148, 2024, Art. no. 109089.
- [24] C. Yan et al., "Collision-avoiding flocking with multiple fixed-wing UAVs in obstacle-cluttered environments: A task-specific curriculum-based MADRL approach," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 8, pp. 10894–10908, Aug. 2024.
- [25] C. Yan, C. Wang, H. Zhou, X. Xiang, X. Wang, and L. Shen, "Multi-agent reinforcement learning with spatial-temporal attention for flocking with collision avoidance of a scalable fixed-wing UAV fleet," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 2, pp. 1769–1782, Feb. 2025.
- [26] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. Adv. Neural Informat. Process. Syst.*, 2020, pp. 6840–6851.
- [27] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. Int. Conf. Learn. Representations*, 2021.
- [28] L. Yang et al., "Policy representation via diffusion probability model for reinforcement learning," 2023, *arXiv:2305.13122*.
- [29] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, "Learning a diffusion model policy from rewards via Q-score matching," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 41163–41182.
- [30] C. Yan et al., "PASCAL: Population-specific curriculum-based MADRL for collision-free flocking with large-scale fixed-wing UAV swarms," *Aerosp. Sci. Technol.*, vol. 133, 2023, Art. no. 108091.
- [31] C. Yan, X. Xiang, and C. Wang, "Fixed-wing UAVs flocking in continuous spaces: A deep reinforcement learning approach," *Robot. Auton. Syst.*, vol. 131, 2020, Art. no. 103594.
- [32] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [33] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6382–6393.
- [34] H. Du et al., "Diffusion-based reinforcement learning for edge-enabled AI-generated content services," *IEEE Trans. Mobile Comput.*, vol. 23, no. 9, pp. 8902–8918, Sep. 2024.
- [35] X. Tang et al., "Task assignment and exploration optimization for low altitude UAV rescue via generative AI enhanced multi-agent reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 25, no. 1, pp. 627–643, Jan. 2026, doi: [10.1109/TMC.2025.3594188](https://doi.org/10.1109/TMC.2025.3594188).
- [36] S. Iqbal and F. Sha, "Actor-attention-critic for multi-agent reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2961–2970.
- [37] J. K. Gupta, M. Egorov, and M. Kochenderfer, "Cooperative multi-agent control using deep reinforcement learning," in *Proc. Int. Conf. Auton. Agents Multiagent Syst.*, 2017, pp. 66–83.
- [38] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.
- [39] D. Misra, "Mish: A self regularized non-monotonic activation function," in *Proc. Brit. Mach. Vis. Conf.*, 2020.
- [40] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [41] R. E. Wang, M. Everett, and J. P. How, "R-MADDPG for partially observable environments and limited communication," 2020, *arXiv:2002.06684*.
- [42] X. Xing, Z. Zhou, Y. Li, B. Xiao, and Y. Xun, "Multi-UAV adaptive cooperative formation trajectory planning based on an improved MATD3 algorithm of deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 12484–12499, Sep. 2024.
- [43] C. Yan, C. Wang, X. Xiang, Z. Lan, and Y. Jiang, "Deep reinforcement learning of collision-free flocking policies for multiple fixed-wing UAVs using local situation maps," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 1260–1270, Feb. 2022.



Chao Yan (Member, IEEE) received the B.E. degree in electrical engineering and automation from the China University of Mining and Technology, Xuzhou, China, in 2017, and the M.S. and Ph.D. degrees in control science and engineering from the National University of Defense Technology, Changsha, China, in 2019 and 2023, respectively.

From 2021 to 2022, he was a Visiting Ph.D. Student with the School of Mechanical and Aerospace Engineering, Nanyang Technological University, Singapore. He is currently an Associate Professor with the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China. His research interests include deep reinforcement learning and cooperative control of AAV swarms.



Yuna Jiang (Member, IEEE) received the B.E. degree in information engineering from the China University of Mining and Technology, Xuzhou, China, in 2017, and the Ph.D. degree in information and communication engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2023.

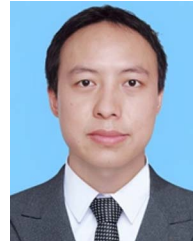
From 2021 to 2022, she was a Visiting Ph.D. Student with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. She is currently an Associate Professor with the School of Communications and Information Engineering, Nanjing University of Posts and Telecommunications, Nanjing, China. Her research interests include multimedia communication, wireless resource allocation, game theory, and metaverse.



Mou Chen (Fellow, IEEE) received the B.Sc. degree in material science and engineering and the Ph.D. degree in automatic control engineering from the Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 1998 and 2004, respectively.

From 2007 to 2008, he was an Academic Visitor with the Department of Aeronautical and Automotive Engineering, Loughborough University, Loughborough, U.K. From 2008 to 2009, he was a Research Fellow with the Department of

Electrical and Computer Engineering, National University of Singapore, Singapore. From May to November 2014, he was a Senior Academic Visitor with the School of Electrical and Electronic Engineering, The University of Adelaide, Adelaide, SA, Australia. He is currently a Professor with the College of Automation Engineering, NUAA. His research interests include nonlinear system control, intelligent control, and flight control.



Xiangke Wang (Senior Member, IEEE) received the B.E., M.E., and Ph.D. degrees in control science and engineering from the National University of Defense Technology (NUDT), Changsha, China, in 2004, 2006, and 2012, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, NUDT. He has authored or coauthored three books and more than 100 publications in peer-reviewed journals and international conferences. His re-

search interests include the control of multiagent systems and their applications on autonomous aerial vehicles.



Xiaojia Xiang received the B.E., M.S., and Ph.D. degrees in automatic control from the National University of Defense Technology (NUDT), Changsha, China, in 2003, 2007, and 2016, respectively.

He is currently a Professor with the College of Intelligence Science and Technology, NUDT. His research interest include mission planning, and cooperative control of autonomous systems.